CaRL - Solving Catan using RL

Group members: Arrush Shah and Suraj Kidiyoor
5/20/2026
Syslab P3: Dr. Yilmaz

Problem

- We want to make our own Catan bot using Reinforcement learning
- Building an AI for Catan is difficult because of the many factors in the game
- We want a flexible algorithm that can improve itself over time
- Lack of current Catan algorithms publicly available

Background

- What is Catan?
  - Victory Points (VP) are the goal of the game
- Traditional game AI does not work well with unpredictable conditions
- Reinforcement learning allows the AI to adapt

Starting Board

Figure 1: #5

Setup/Starting

Figure 1: #5

After setup phase is complete + First turn

Figure 1: #5

Building roads/settlements

Figure 1: #5

Rolling a 7, what is a robber?

Figure 1: #5

Trading

Figure 1: #5

Development Cards

Development Cards have special abilities. There are 25 total in the deck and they can be played before the dice is rolled.

14 Knight cards	Move the Robber to another tile and steal one random resource from a player on that tile
2 Road Building cards	Place 2 roads for free
2 Year of Plenty cards	Select 2 resources of your choice from the bank
2 Monopoly cards	Steal all resources of one type from all other players
5 +1 point cards	Player secretly gets awarded 1 Point. +1 Point cards are revealed once you reach 10 points.

Figure 1: #5

Victory points and ending the game

The first player that gets to 10 Points is the winner. Get points by doing the following:

+1 point	Build a settlement
+1 point	Upgrade a settlement to a city
+1 point	Buy Development Cards for a chance to get a +1 Point Card
+2 points	Get the Longest Road. A minimum of 5 consecutive roads is required to get this achievement
+2 points	Get the Largest Army. Playing a minimum of 3 Knight Development Cards is required to get this achievement

Figure 1: #5
Figure 2: #5

OTHER SOLUTIONS

- Rule-based bots
- Search-based methods
- Supervised Learning

Rule Based Bots

- JAVA GUI (JSettlers2)
- Although up to date, not adaptable

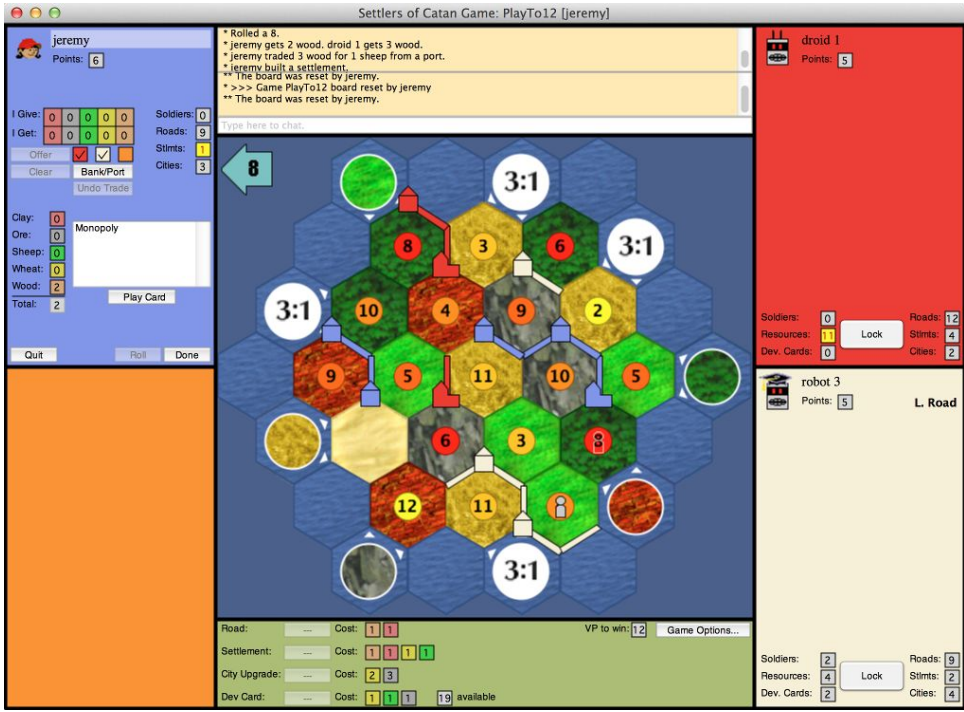


Figure 1: #6

Search Based Methods

- Implementations utilize Alpha-Beta, as well as A-Star
- High computing cost
- Non-deterministic game (dice rolls)

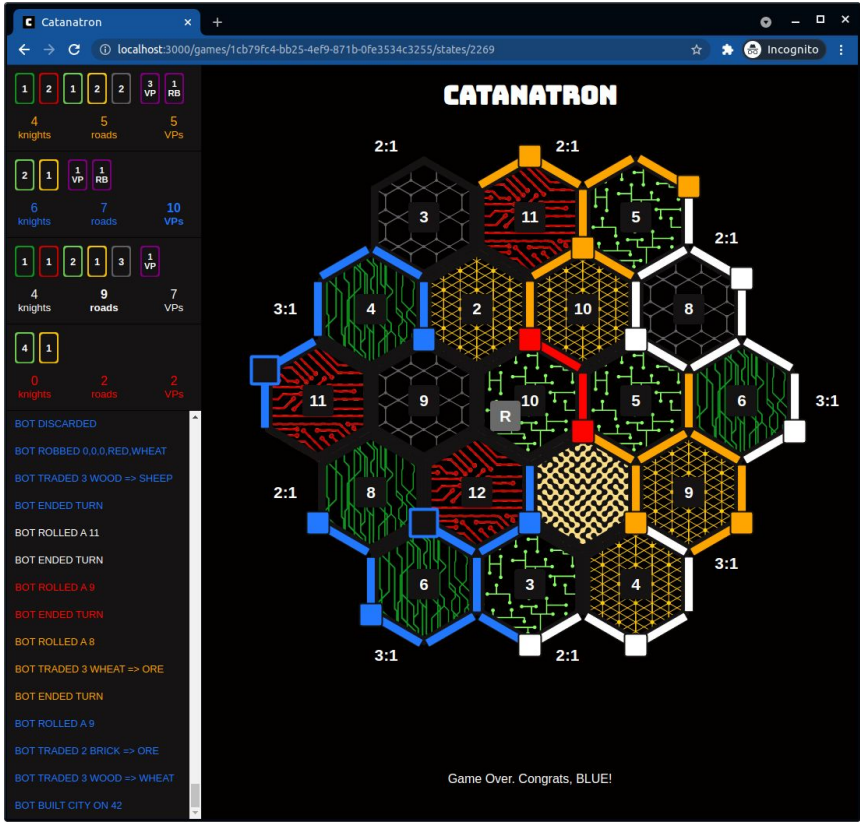


Figure 1: #7

Supervised Learning

- Not viable for Catan because of lack of datasets
- Catanatron implements this somewhat, but a dependence on it can lead to overfitting

Impact

- Advances in a field that has seen no recent updates (Most popular results haven’t been updated in 5 years)
- Progress research into game theory
- Allow people to play against stronger and smarter Catan bots to help them train and learn

What is Reinforcement Learning?

Reinforcement learning is a framework where an agent:

1. Observes a state
2. Takes an action
3. Receives a reward
4. Transitions to a new state
5. Uses that experience to improve future decisions

PACKAGES

Used to host the environment of the game and get the data from it.

NumPy

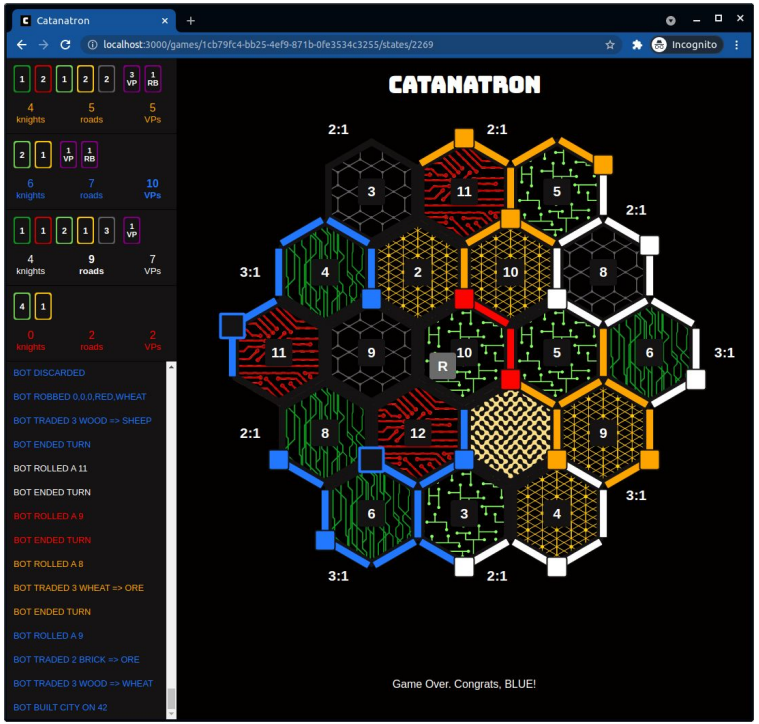
Used for most scientific calculations on python, and from C++ (which makes it faster)

PACKAGES

Stable_baselines3

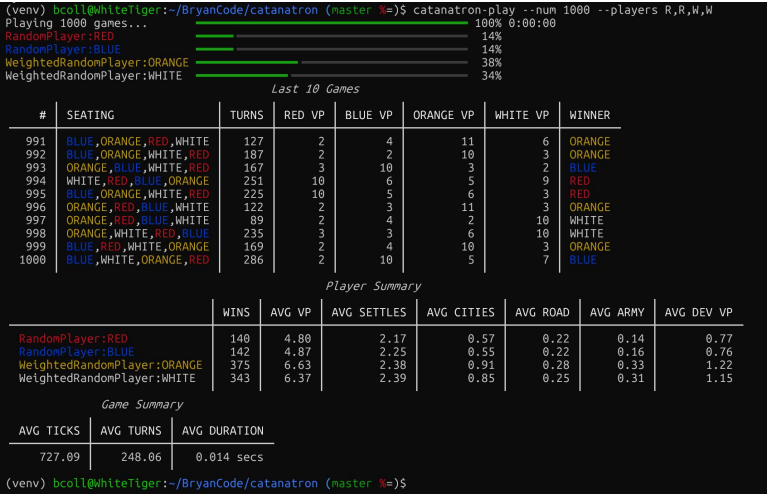
Arguably the most important package. Creates the reinforcement learning behind the robot, and allows for simple implementation and customization of variables (such as reward, and training time)

ENVIRONMENT BREAKDOWN



The environment is from the creators of catanatron (https://docs.catanatron.com/) and their environment and bots are going to be used to train our model.

ENVIRONMENT BREAKDOWN



```
(venv) bcoll@WhiteTiger:~/BryanCode/catanatron (master %=)$ catanatron-play --num 1000 --players R,R,W,W
Playing 1000 games...                                                          100% 0:00:00
RandomPlayer:RED                                                                14%
RandomPlayer:BLUE                                                               14%
WeightedRandomPlayer:ORANGE                                                     38%
WeightedRandomPlayer:WHITE                                                      34%
```

Last 10 Games

#	SEATING	TURNS	RED VP	BLUE VP	ORANGE VP	WHITE VP	WINNER
991	BLUE,ORANGE,RED,WHITE	127	2	4	11	6	ORANGE
992	BLUE,ORANGE,WHITE,RED	187	2	2	10	3	ORANGE
993	ORANGE,BLUE,WHITE,RED	167	3	10	3	2	BLUE
994	WHITE,RED,BLUE,ORANGE	251	10	6	5	9	RED
995	BLUE,ORANGE,WHITE,RED	225	10	5	6	3	RED
996	ORANGE,RED,BLUE,WHITE	122	2	3	11	3	ORANGE
997	ORANGE,RED,BLUE,WHITE	89	2	4	2	10	WHITE
998	ORANGE,WHITE,RED,BLUE	235	3	3	6	10	WHITE
999	BLUE,RED,WHITE,ORANGE	169	2	4	10	3	ORANGE
1000	BLUE,WHITE,ORANGE,RED	286	2	10	5	7	BLUE

Player Summary

	WINS	AVG VP	AVG SETTLES	AVG CITIES	AVG ROAD	AVG ARMY	AVG DEV VP
RandomPlayer:RED	140	4.80	2.17	0.57	0.22	0.14	0.77
RandomPlayer:BLUE	142	4.87	2.25	0.55	0.22	0.16	0.76
WeightedRandomPlayer:ORANGE	375	6.63	2.38	0.91	0.28	0.33	1.22
WeightedRandomPlayer:WHITE	343	6.37	2.39	0.85	0.25	0.31	1.15

Game Summary

AVG TICKS	AVG TURNS	AVG DURATION
727.09	248.06	0.014 secs

```
(venv) bcoll@WhiteTiger:~/BryanCode/catanatron (master %=)$
```

Reasoning for use:

- Backend with gymnasium
- Allows integration of stable_baselines3 to train the model with their simulation ability
- In the future, we can run testing sessions with their different bots, as shown to the left

METHODS

The process begins by passing the current environment state to the model

The state represents the full Catan game situation:

- Board configuration
- Player resources and victory points
- Turn and action constraints

Stable-Baselines3 automatically:

- Converts the environment output into a readable format
- Normalizes and flattens the observation if needed

This encoded state becomes the input vector to the neural network

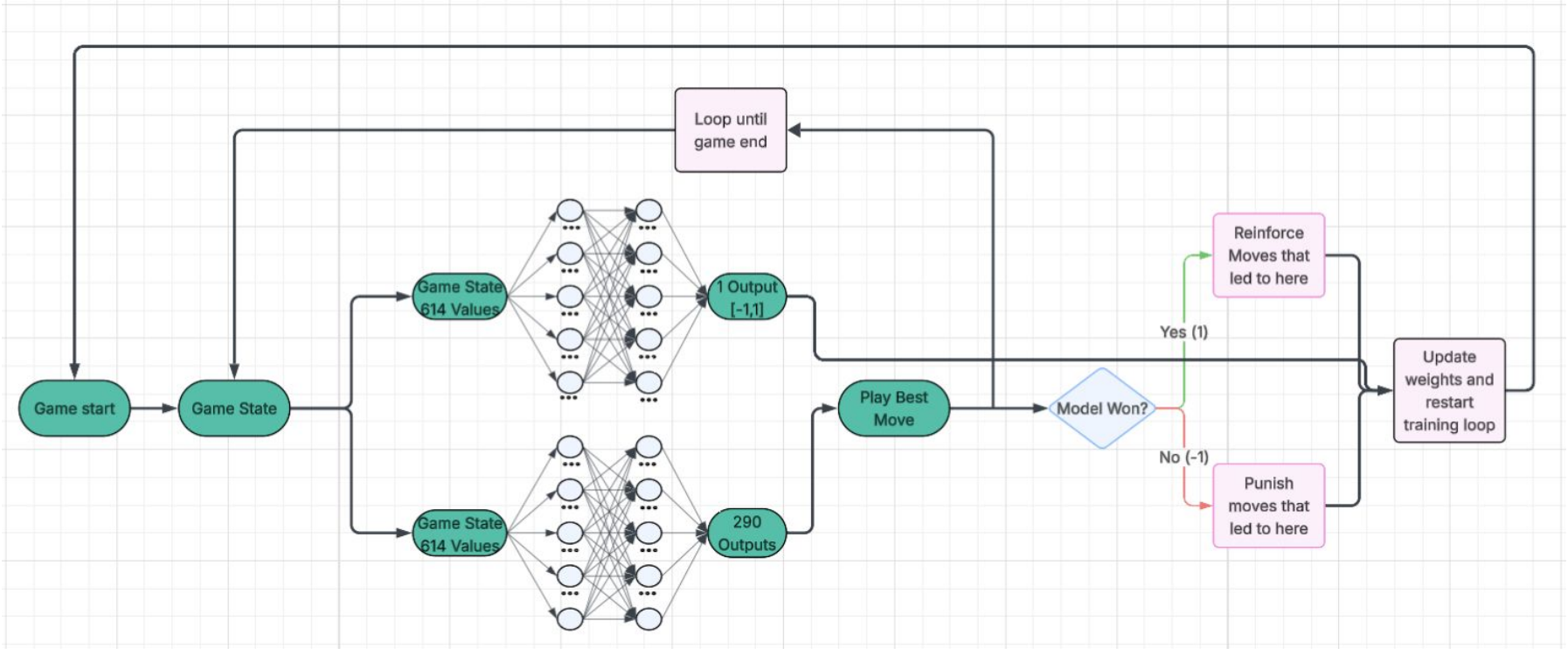
THE REINFORCEMENT BACKEND

The model uses MaskablePPO

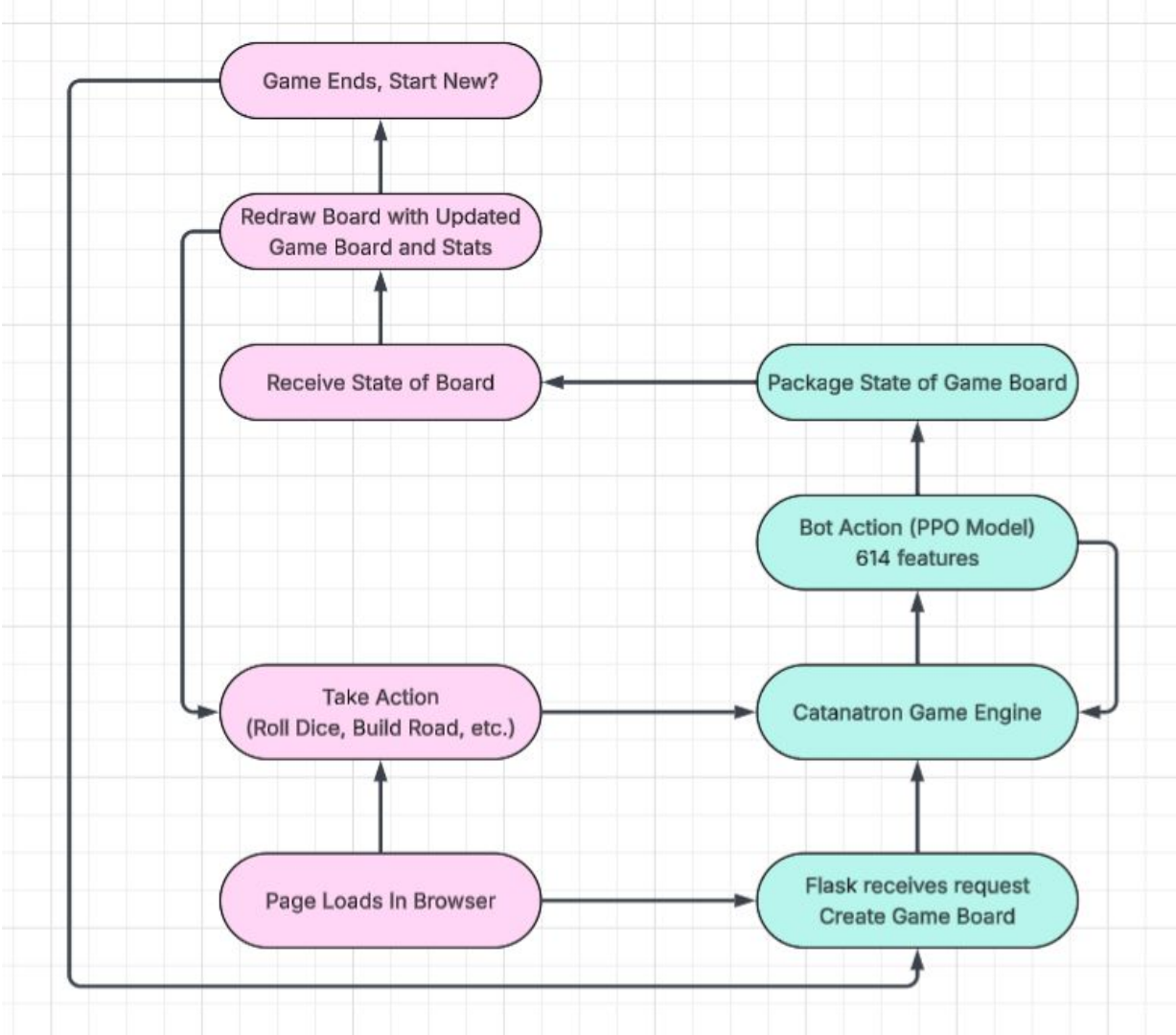
The neural network consists of:

- Shared hidden layers or Neural Network weights
- Two separate outputs:
  - Move valuation (higher score means better move)
  - Game state (predicts a win, or a loss -1 or 1)
- The “Mask” portion of the PPO removes the illegal moves from the move valuations

OVERALL ARCHITECTURE



Catan Website



BASIC REWARD FUNCTION

- By training the model, currently with this simple reward function we have seen 0 improvement (and just noise) with minimal training
- After 81 seconds of training the model seems to be equivalent to randomly choosing a move to play

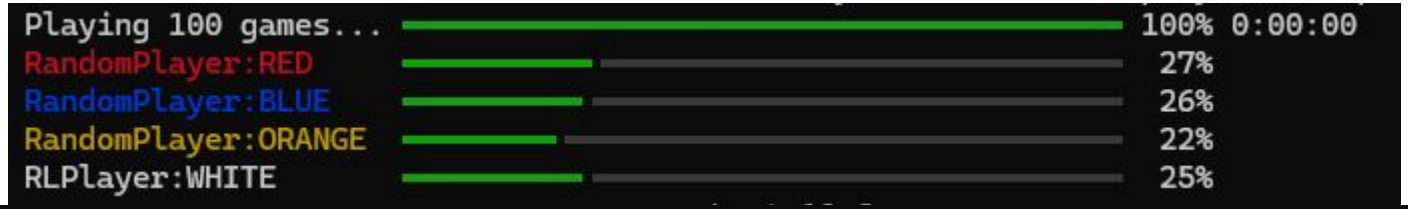
```python
def vp_reward(game, p0_color):
    winning_color = game.winning_color()
    if winning_color is None:
        return 0.0
    elif winning_color == p0_color:
        return 1.0
    else:
        return -1.0
```

```
rollout/                |          |
   ep_len_mean          | 460      |
   ep_rew_mean          | 0        |
time/                   |          |
   fps                  | 1224     |
   iterations           | 49       |
   time_elapsed         | 81       |
   total_timesteps      | 100352   |
```

MODEL TESTING

Model after 5 million steps (1 hour of training) over 100 games

No significant change between random



Player Summary

	WINS	AVG VP	AVG SETTLES	AVG CITIES	AVG ROAD	AVG ARMY	AVG DEV VP
RandomPlayer:RED	27	5.66	2.19	0.71	0.30	0.21	1.03
RandomPlayer:BLUE	26	5.94	2.40	0.69	0.29	0.25	1.08
RandomPlayer:ORANGE	22	5.86	2.39	0.68	0.27	0.24	1.09
RLPlayer:WHITE	25	5.71	1.70	1.13	0.12	0.29	0.93

MODEL TESTING

Model after 20 million steps (4 hours of training) over 100 games

	WINS	AVG VP	AVG SETTLES	AVG CITIES	AVG ROAD	AVG ARMY	AVG DEV VP
RandomPlayer:RED	10	4.05	2.02	0.48	0.18	0.05	0.61
RandomPlayer:BLUE	11	4.67	2.26	0.52	0.26	0.10	0.65
RandomPlayer:ORANGE	12	4.54	2.25	0.40	0.25	0.12	0.75
RLPlayer:WHITE	67	8.66	2.20	1.35	0.27	0.70	1.82

FAILED REWARD FUNCTIONS

- Attempt at LongTermMemory
  - Kept track of
    - public victory points, actual victory points, whether they have longest road, and whether they have largest army
  - Mid-game reward
    - Each step subracts reward by 0.01
    - + 1.5 times change in VP
    - + 0.75 plus for longest road (-0.75 for losing it)
    - + 0.75 plus for largest army (0.75 for losing it)
    - ± 20 points for losing/winning

FAILED REWARD FUNCTIONS

- Third attempt, after failure with a complex reward function (
  - + 0.2 times change in VP
  - Win adds 10 points
  - Loss subracts 10 points

```
reward = 0.2 * (vp - self.prev_vp)

winner = game.winning_color()
if winner is not None:
    reward += 10.0 if str(winner) == str(p0_color) else -10.0
```

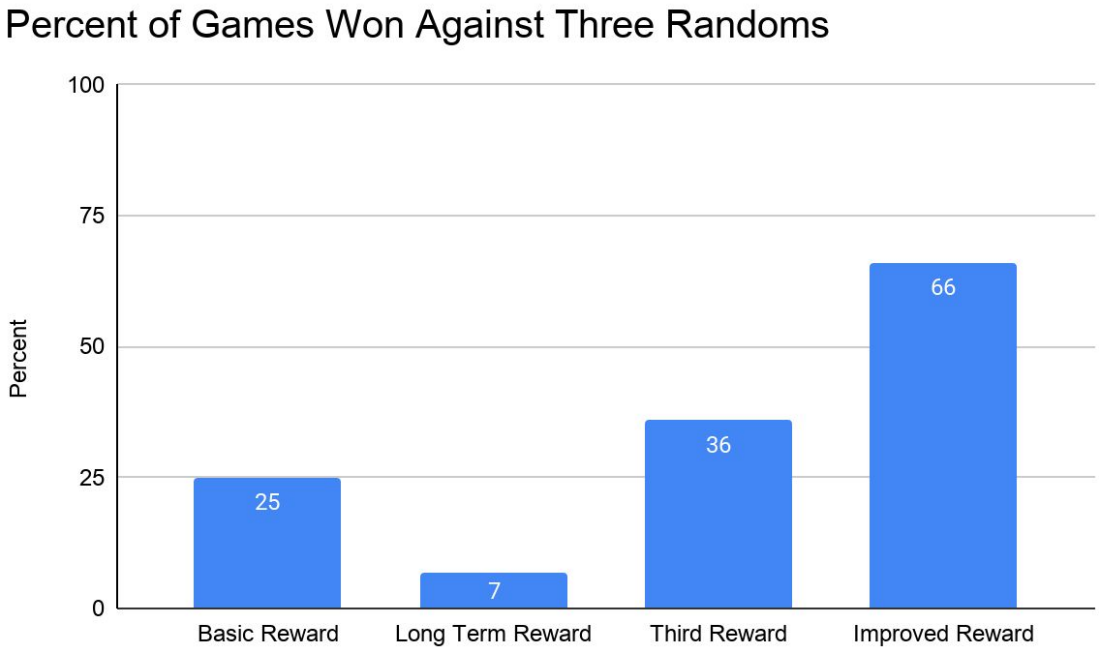
IMPROVED REWARD FUNCTION

- Win ➔ + 10
- Loss ➔ - 10
  - Other rewards for during games
    - +2.5 times change in VP
    - +1.5 times change in settlements
    - +2.5 times change in cities
- Overall this new reward function is trying to keep the code relatively simple while keeping functionality

```
reward = 0.0
reward += 2.5 * (vp - self.prev_vp)
reward += 1.5 * (settlements - self.prev_settlements)
reward += 2.5 * (cities - self.prev_cities)
```

COMPARING REWARD FUNCTIONS

Trials done with a baseline of 1 hour of training with 100 games against three randoms.



MODEL TRAINING/TESTING WITH HARDER MODELS

AI Leaderboard given by Catanatron

Player	% of wins in 1v1 games	num games used for result
AlphaBeta(n=2)	80% vs ValueFunction	25
ValueFunction	90% vs GreedyPlayouts(n=25)	25
GreedyPlayouts(n=25)	100% vs MCTS(n=100)	25
MCTS(n=100)	60% vs WeightedRandom	15
WeightedRandom	60% vs Random 50% vs VictoryPoint	1000
VictoryPoint	60% vs Random	1000
Random	-	-

THE TRAINING PROCESS

1. Trained against 3 random bots for 10 million steps (about 2 hours)

2. Trained against 1 weighted random bot, and 2 random bots for 6 million steps (about 1 hour)

3. Trained against 3 weighted random bots for 2 million steps (<1 hour)

4. Trained against 1 ValueFunction bot and 2 weighted random bots for 2 million steps (about 4 hours)

5. Trained against 2 ValueFunction bots and 1 weighted random bot for 2 million steps (about 4 hours)

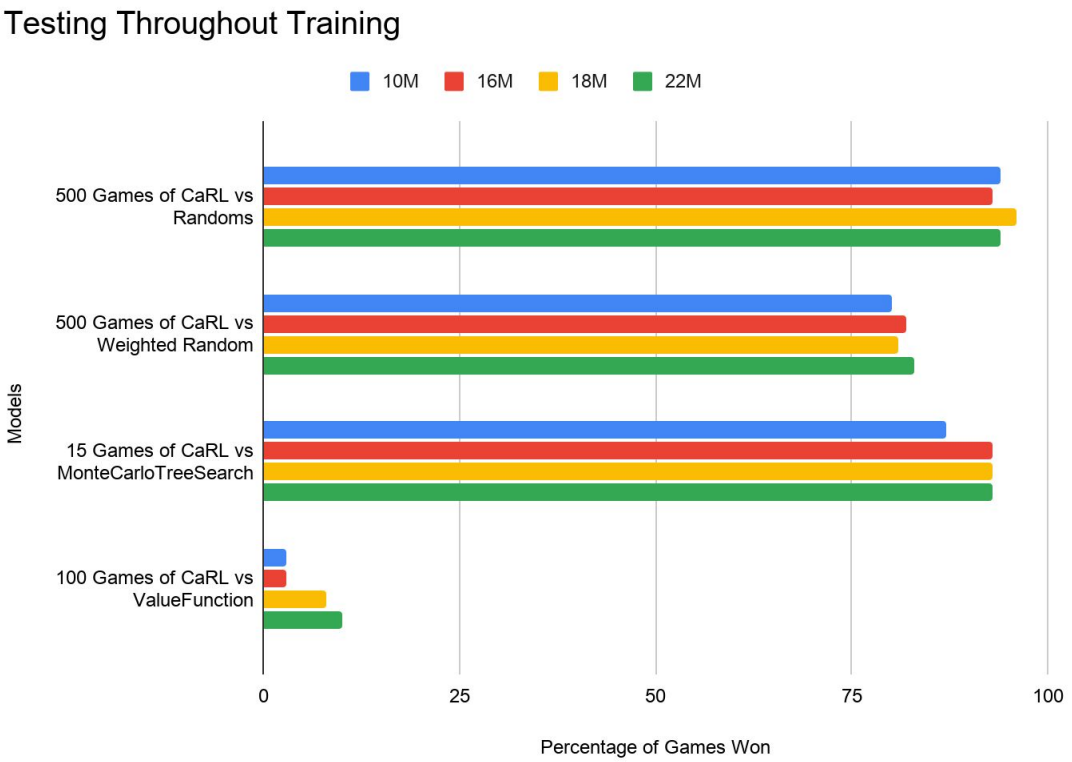
THE TRAINING PROCESS

- Why does ValueFunction training take so long?

- Since we ask that bot to evaluate its board and make a move at every single turn, the computation required heavily slows down training.

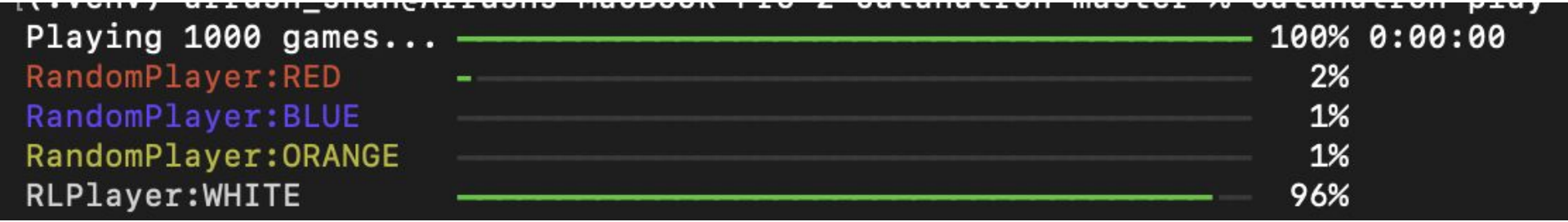
- Compared to WeightedRandom and Random, its 5 times slower per ValueFunction bot when training against it.

TESTING THE MODEL THROUGH TRAINING

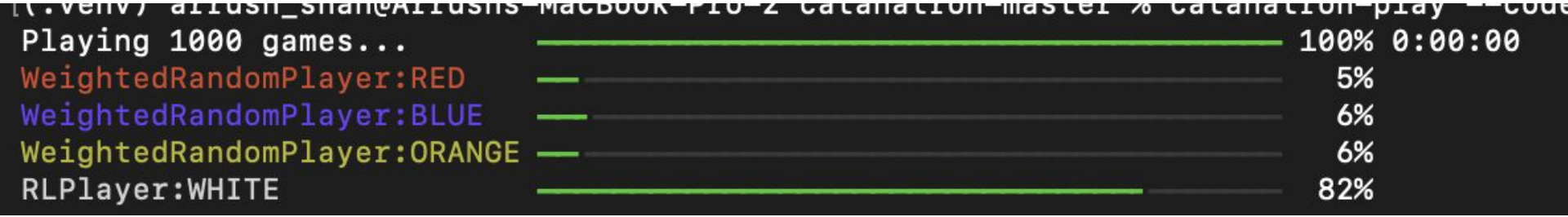


TESTING THE MODEL

1000 Games against Random.

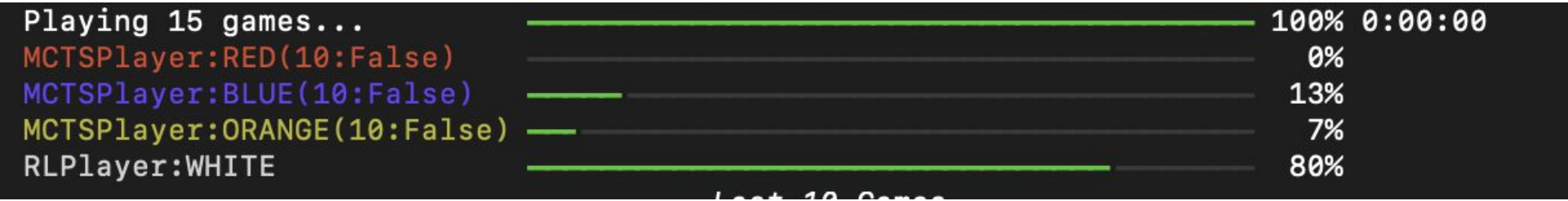


1000 games against WeightedRandom

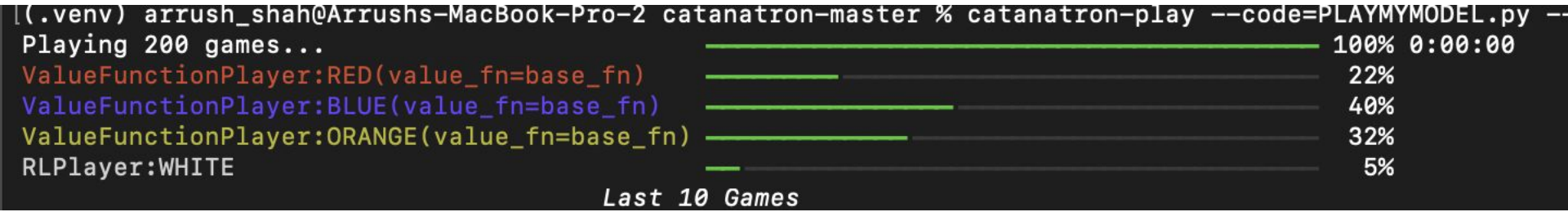


TESTING THE MODEL

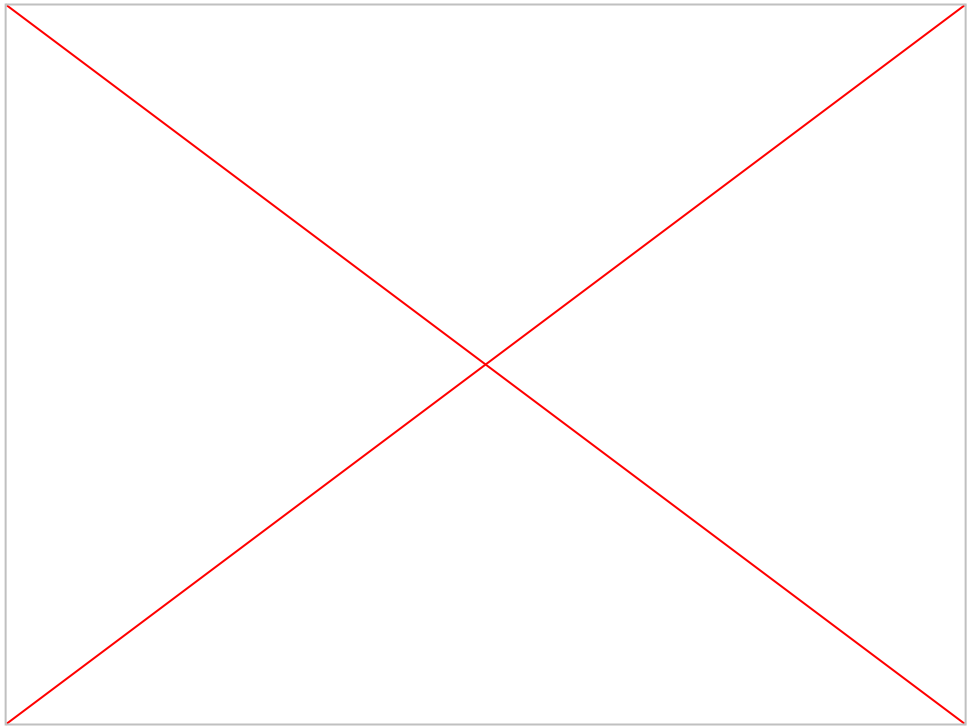
Against MCTS. (given to have a 60% winrate against WeightedRandom).



Against ValueFunction (given to have a ~100% winrate against MCTS)

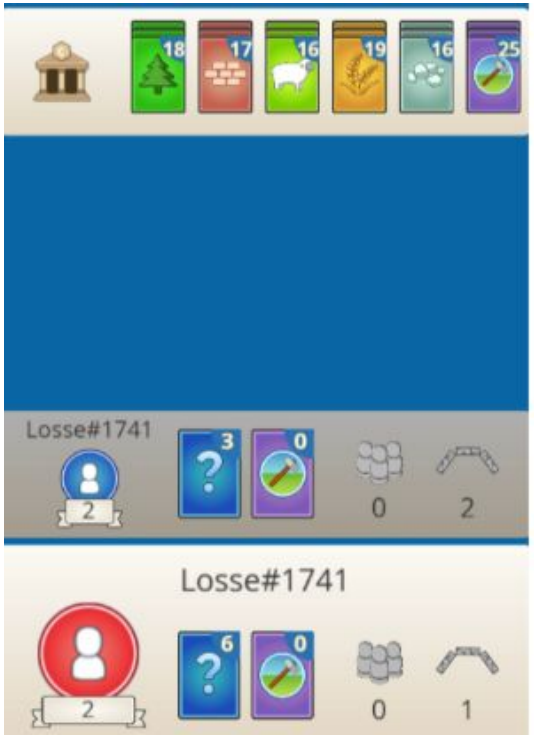


WEBSITE DEMO



Why IS Ours BETTER?

- Our model is significantly faster to evaluate a move compared to stronger models, like MonteCarloTreeSearch and ValueFunction
- Our model prioritizes cities, which give double the win points over settlements, allowing for easier wins



Limitations

- Training the model
  - Training takes a long time
  - Opponent bot speed is a huge factor in training
- Website doesn’t allow for training of the model
- There is no guarantee that more training will lead to linearly increasing accuracy against ValueFunction (although that is currently observed)

RESULTS

Reward design had a major effect on performance.

In one-hour tests against random bots:

- Basic reward: 25% win rate
- Long-term reward: 7% win rate
- Third reward: 36% win rate
- Improved reward: 66% win rate

Longer training improved performance against simple bots.

- Final model performed strongly against Random, WeightedRandom, and MCTS bots.
- Model won about 80% against MCTS, but only 5% against ValueFunction.
- Overall, the model improved beyond random play but still struggled against stronger strategy-based bots.

CONCLUSION AND FUTURE WORK

Our model can be placed right below ValueFunction, since it outperforms all of the below algorithms.

Player	% of wins in 1v1 games	num games used for result
AlphaBeta(n=2)	80% vs ValueFunction	25
ValueFunction	90% vs GreedyPlayouts(n=25)	25
GreedyPlayouts(n=25)	100% vs MCTS(n=100)	25
MCTS(n=100)	60% vs WeightedRandom	15
WeightedRandom	60% vs Random 50% vs VictoryPoint	1000
VictoryPoint	60% vs Random	1000
Random	-	-

CONCLUSION AND FUTURE WORK

- Our website is currently being hosted online using Render web services
- Although it is hosted online, there can only be one game playing at a time

Future Work:

- Train the model more on a stronger setup to speed up training time
- Implement Trading with the opponent instead of just the bank

REFERENCES

1. https://settlers-rl.github.io/
2. https://github.com/ikostrikov/pytorch-a2c-ppo-acktr-gail
3. https://github.com/pytorch/rl
4. https://github.com/henrycharlesworth/settlers_of_catan_RL
5. https://colonist.io
6. https://github.com/jdmonin/JSettlers2
7. https://github.com/bcollazo/catanatron
8. https://docs.pytorch.org/tutorials/beginner/basics/buildmodel_tutorial.html
9. https://github.com/bcollazo/catanatron
10. https://docs.catanatron.com/
11. https://docs.catanatron.com/using-catanatron/interactive-blocks
12. https://docs.catanatron.com/advanced/openapi
13. https://gymnasium.farama.org/
14. https://stable-baselines3.readthedocs.io/
15. https://sb3-contrib.readthedocs.io/
16. https://github.com/Zawiop/canatron-syslab
17. https://github.com/professorcool09/professorcool09.github.io

THANKS!

Q&A